TP Integrador DAW1 · Enunciados y extras

Nov 3, 2026 · @german federico nicolas torres

El trabajo práctico

Diseño de Aplicaciones Web I · Tecnicatura Universitaria en Desarrollo Web · UNdeC

Vas a construir una API REST completa con Spring Boot, como la del sistema de turnos que hicimos en clase, para **uno** de los cinco enunciados de este documento. Además de lo obligatorio, cada enunciado tiene **extras opcionales** que suben la nota.

Modalidad	**Individual**
Entrega	**Domingo 15/11/2026, 23:59.** Se corrige el último commit en `main` anterior a esa hora.
Defensa	**Martes 17/11/2026**, en clase
Cómo se entrega	Repositorio de GitHub creado desde la plantilla de la cátedra, con el link cargado en el aula virtual
Tecnologías	Java 21, Spring Boot 4, Spring Data JPA, H2, Bean Validation

Para empezar

1. Crear la cuenta de GitHub (con verificación en dos pasos) y configurar Git en tu PC.
2. Abrir la plantilla de la cátedra → **Use this template** → repo público llamado `tp-daw1-apellido`.
3. Clonarlo en IntelliJ y generar el proyecto Spring Boot (Web, Data JPA, H2, Validation) **en la raíz** del repo.
4. Escribir la palabra de tu enunciado en la última línea de `.tp/enunciado.txt`.
5. Commit y push. En la pestaña **Actions** de GitHub vas a ver la corrección automática con tu puntaje.
6. Cargar el link del repo en el aula virtual.

Evaluación y puntos extra

La nota va de 1 a 10.

Contrato común (obligatorio en los cinco)

Las pruebas automáticas llaman a tu API por HTTP. Si una ruta, el nombre de un campo o un código de respuesta no coincide, la prueba falla aunque tu código “funcione”.

- Todas las rutas empiezan con `/api`; los recursos van en plural y minúscula.
- Los campos JSON se llaman **exactamente** como en el enunciado (camelCase). Todo recurso devuelve `id`.

- Fechas en formato ISO: `2026-11-20` y `2026-11-20T18:00:00`. Puerto 8080.
- El estado inicial lo pone el sistema; no se manda en el POST.

Operación	Éxito	Errores
`POST /api/recurso`	201 + header `Location: /api/recurso/{id}` + el objeto creado	400 datos inválidos · 404 si un id relacionado no existe · 409 regla de negocio
`GET /api/recurso`	200 + lista (array)	—
`GET /api/recurso/{id}`	200 + el objeto	404 no existe · 400 si el id no es un número
`PUT /api/recurso/{id}`	200 + el objeto modificado	400 · 404 · 409
`DELETE /api/recurso/{id}`	204 sin cuerpo	404 · 409 si no se puede borrar
PATCH `/api/recurso/{id}/estado` con `{"estado":"X"}`	200 + el objeto	400 sin estado o estado inválido · 404 · 409 transición no permitida

Los errores siempre van en formato `ProblemDetail` (`status`, `title`, `detail`). En los 400 de validación, además, un mapa `errores` con campo → mensaje:

```
{ "status": 400, "title": "Datos inválidos", "detail": "La petición tiene errores",
  "errores": { "titulo": "no debe estar vacío" } }
```

Un JSON mal formado también tiene que devolver 400, nunca 500.

Enunciado 1 · Biblioteca — `biblioteca`

Una biblioteca barrial quiere dejar el cuaderno y registrar sus préstamos con un sistema.

Recursos

- `/api/libros`: `titulo` (obligatorio), `autor` (obligatorio), `isbn` (obligatorio y **único**). CRUD completo.
- `/api/socios`: `nombre` (obligatorio), `email` (email válido).
- `/api/prestamos`: `libroId`, `socioId`, `fechaDevolucion` (fecha **futura**). Se crea en estado `ACTIVO`.

Estados del préstamo: `ACTIVO → DEVUELTO | VENCIDO` · `VENCIDO → DEVUELTO` · `DEVUELTO` es final.

Reglas de negocio (409)

1. Un libro con un préstamo no devuelto no se puede volver a prestar.
2. Un socio puede tener como máximo **3** préstamos no devueltos.
3. No puede haber dos libros con el mismo ISBN.

Consultas

- `GET /api/socios/{id}/prestamos`: los préstamos del socio (404 si el socio no existe).
- `GET /api/prestamos?estado=ACTIVO`: los préstamos en ese estado.

Extras de este enunciado

Extra	Qué hay que hacer	Suma
Multa por atraso	Al pasar a `DEVUELTO` después de `fechaDevolucion`, el préstamo guarda `fechaRealDevolucion` y `multa` = días de atraso × $500	+0,5
Socio suspendido	Un socio con una multa sin pagar no puede pedir préstamos (409). `PATCH /api/prestamos/{id}/pagar-multa` la marca como pagada	+0,5
Disponibilidad	`GET /api/libros?disponible=true` devuelve solo los libros sin préstamo activo o vencido	+0,25
Ranking	`GET /api/libros/mas-prestados?top=5` devuelve los libros con más préstamos y la cantidad de cada uno	+0,25
Renovación	`PATCH /api/prestamos/{id}/renovar` suma 7 días a `fechaDevolucion`, una sola vez y solo si está `ACTIVO` (si no, 409)	+0,25

Enunciado 2 · Gimnasio — `gimnasio`

Un gimnasio quiere que sus socios se inscriban a las clases sin superar el cupo de cada una.

Recursos

- `/api/actividades`: `nombre` (obligatorio), `fechaHora` (fecha y hora **futura**), `cupo` (entero **positivo**). CRUD completo.
- `/api/socios`: `nombre` (obligatorio), `email` (email válido).
- `/api/inscripciones`: `actividadId`, `socioId`. Se crea en estado `CONFIRMADA`.

Estados de la inscripción: `CONFIRMADA → ASISTIO | CANCELADA` · los dos son finales.

Reglas de negocio (409)

1. No hay lugar si las inscripciones **no canceladas** ya llegan al cupo.
2. Un socio no puede inscribirse dos veces a la misma actividad (salvo que la anterior esté cancelada).
3. No se puede cancelar si faltan **menos de 2 horas** para la actividad.

Consultas

- `GET /api/actividades/{id}/inscripciones`: las inscripciones de la actividad (404 si no existe).
- `GET /api/actividades?fecha=2026-11-20`: las actividades de ese día.

Extras de este enunciado

Extra	Qué hay que hacer	Suma
Lista de espera	`POST /api/actividades/{id}/lista-espera` con `{"socioId":…}` anota al socio cuando el cupo está lleno. Al cancelarse una inscripción, el primero de la lista queda inscripto automáticamente	+0,5
Lugares libres	Cada actividad devuelve `lugaresDisponibles`, calculado por el sistema (no se guarda)	+0,25
Asistencia del socio	`GET /api/socios/{id}/asistencia` devuelve inscripciones totales, asistencias y el porcentaje	+0,25
Clases semanales	`POST /api/actividades/semanales` con los datos de la actividad y `semanas: N` crea la misma clase N semanas seguidas	+0,5
Ranking	`GET /api/actividades/mas-concurridas?top=5`: las actividades con más asistencias	+0,25

Enunciado 3 · Taller mecánico — `taller`

Un taller quiere seguir cada auto desde que entra hasta que se lo entrega al cliente.

Recursos

- `/api/clientes`: `nombre` (obligatorio), `telefono` (obligatorio).
- `/api/vehiculos`: `patente` (formato `ABC123` o `AB123CD`, **única**), `marca` y `modelo` (obligatorios), `clienteId`. CRUD completo.
- `/api/ordenes`: `vehiculoId`, `descripcion` (obligatoria). Se crea en estado `INGRESADO`. `PUT /api/ordenes/{id}` permite cargar `descripcion` y `costoFinal`.

Estados de la orden: `INGRESADO → EN_REPARACION | CANCELADO` · `EN_REPARACION → TERMINADO` · `TERMINADO → ENTREGADO`.

Reglas de negocio (409)

1. Un vehículo con una orden **abierta** (ni entregada ni cancelada) no puede tener otra.
2. No puede haber dos vehículos con la misma patente.
3. No se puede pasar a `ENTREGADO` sin `costoFinal` cargado.

Consultas

- `GET /api/clientes/{id}/vehiculos`: los vehículos del cliente (404 si no existe).
- `GET /api/ordenes?estado=EN_REPARACION`: las órdenes en ese estado.

Extras de este enunciado

Extra	Qué hay que hacer	Suma
Repuestos con stock	`/api/repuestos` (`nombre`, `precio`, `stock`) y `POST /api/ordenes/{id}/repuestos` con `{"repuestoId":…,"cantidad":…}`: descuenta stock (409 si no alcanza)	+0,5
Costo calculado	Si la orden tiene repuestos, devuelve `costoRepuestos` (suma de precio × cantidad) y `costoFinal` no puede ser menor (400)	+0,25
Historial por patente	`GET /api/vehiculos/patente/{patente}/historial`: todas las órdenes del vehículo, de la más nueva a la más vieja	+0,25
Facturación mensual	`GET /api/ordenes/facturacion?mes=2026-11`: cantidad de órdenes entregadas y total facturado en ese mes	+0,5
Tiempo de reparación	Cada orden guarda la fecha y hora de cada cambio de estado y devuelve `horasEnReparacion` una vez terminada	+0,25

Enunciado 4 · Complejo deportivo — `canchas`

Un complejo de canchas quiere tomar reservas por hora sin que dos equipos se encuentren en la misma cancha.

Recursos

- `/api/canchas`: `nombre` y `deporte` (obligatorios), `precioHora` (**positivo**). CRUD completo.
- `/api/clientes`: `nombre` (obligatorio), `telefono` (obligatorio).
- `/api/reservas`: `canchaId`, `clienteId`, `inicio` (fecha y hora **futura**), `horas` (entre **1 y 3**). Se crea en estado `PENDIENTE`. El sistema calcula `precioTotal = precioHora × horas`.

Estados de la reserva: `PENDIENTE → CONFIRMADA | CANCELADA` · `CONFIRMADA → FINALIZADA | CANCELADA`.

Reglas de negocio (409)

1. Dos reservas **no canceladas** de la misma cancha no se pueden superponer. Una que empieza justo cuando termina otra es válida.

Consultas

- `GET /api/canchas/{id}/reservas?fecha=2026-11-20`: las reservas de esa cancha ese día (404 si la cancha no existe).
- `GET /api/clientes/{id}/reservas`: las reservas del cliente.

Extras de este enunciado

Extra	Qué hay que hacer	Suma

Extra	Qué hay que hacer	Suma
Horarios libres	`GET /api/canchas/{id}/disponibilidad?fecha=2026-11-20`: las horas libres entre las 8:00 y las 23:00 de ese día	+0,5
Tarifa nocturna	Las horas que empiezan a las 20:00 o después cuestan un 20 % más; `precioTotal` lo tiene en cuenta	+0,25
Turno fijo	`POST /api/reservas/fijas` con los datos de la reserva y `semanas: N` crea la misma reserva N semanas seguidas; si alguna se superpone, no crea ninguna (409)	+0,5
Cancelación tardía	Cancelar con menos de 24 h de anticipación deja la reserva con `recargo` = 50 % de `precioTotal`	+0,25
Recaudación	`GET /api/canchas/recaudacion?mes=2026-11`: total facturado por cancha con reservas finalizadas	+0,25

Enunciado 5 · Eventos — `eventos`

Una productora quiere vender entradas para sus eventos sin vender más de las que entran en la sala.

Recursos

- `/api/eventos`: `nombre` (obligatorio), `fecha` (fecha y hora **futura**), `capacidad` (**positiva**), `precio` (0 o más). CRUD completo.
- `/api/asistentes`: `nombre` (obligatorio), `email` (email válido).
- `/api/entradas`: `eventoId`, `asistenteId`. Se crea en estado `RESERVADA`.

Estados de la entrada: `RESERVADA → PAGADA | ANULADA` · `PAGADA → USADA | ANULADA`.

Reglas de negocio (409)

1. No se venden más entradas **no anuladas** que la capacidad del evento.
2. Un asistente no puede tener dos entradas activas para el mismo evento.
3. Un evento que tiene entradas no se puede borrar.

Consultas

- `GET /api/eventos/{id}/entradas`: las entradas del evento (404 si no existe).
- `GET /api/eventos?nombre=rock`: los eventos cuyo nombre contiene ese texto, sin distinguir mayúsculas.

Extras de este enunciado

Extra	Qué hay que hacer	Suma
Código de entrada	Cada entrada recibe un `codigo` único (UUID). `POST /api/entradas/validar` con `{"codigo":…}` la pasa a `USADA`; 409 si no está pagada o ya se usó	+0,5

Extra	Qué hay que hacer	Suma
Preventa	Las entradas pagadas más de 7 días antes del evento cuestan un 20 % menos; cada entrada devuelve el `importe` que pagó	+0,25
Ocupación	`GET /api/eventos/{id}/ocupacion`: vendidas, disponibles y porcentaje ocupado	+0,25
Transferencia	`PATCH /api/entradas/{id}/transferir` con `{"asistenteId":…}` pasa una entrada pagada a otro asistente, respetando la regla 2	+0,5
Anulación automática	Una entrada `RESERVADA` que no se pagó en 48 h se anula sola (con `@Scheduled` o al consultarla) y libera el lugar	+0,25

Antes de entregar

- ☐ La palabra de mi enunciado está en la última línea de `.tp/enunciado.txt`.
- ☐ El último push está antes del domingo 15/11 a las 23:59 y la corrida de Actions terminó.